

27. Linked lists

Herman Kamper

This is the start of the last big topic of the course: dynamic data structures. It has a reputation for being tricky, and students do sometimes struggle with it. I don't think it is actually difficult. What makes it feel hard is that it brings together three things you have already seen: structures, pointers and `malloc`. If you are comfortable with all three, then linked lists are just a clever way of combining them. If you are not, linked lists will expose exactly where the gaps are.

In this lecture we meet the simplest dynamic data structure, the linked list. We will see how a node is defined, how nodes are strung together, and how to search a list, insert into it and delete from it. Most of the lecture is spent stepping through pictures, because the pictures are what make the code make sense.

Motivation: lists that grow and shrink

So far we have stored a collection of things in one of two ways. Either we declare an array, or, if we are a bit fancier, we reserve a block of memory with `malloc` and treat it as an array:

```
int array[100];  
int *otherArray = malloc(numElements * sizeof(int));
```

In both cases the size is fixed once the block has been reserved. With the array, the size is known before the program even runs. With `malloc`, the size can be decided while the program runs, but once the operating system has handed you the block, you cannot make it bigger or smaller. (You can ask for a new, bigger block and copy everything across, but that is extra work that you have to do yourself.)

In many real situations this is not what you want. Imagine you are a bank. When you open your doors, you have a handful of customers; over time the list of customers grows. If you are a bad bank, you might lose customers, and then the list should shrink. Or imagine a robot moving through a maze, where you want to remember every square it has visited, and there is no limit on how many steps it can take. In both cases the number of elements changes while the program runs.

One option is to define an array that is enormous, say a million elements, and hope it is enough. But that blocks off a huge amount of memory for no reason, just in case you ever get a million customers. A better solution is a *dynamic data structure*: one that grows and shrinks as you add and remove things.

Most programs you use in practice work like this. If you create a spreadsheet with 100 rows, it

does not store the million rows you could scroll down to; it stores the 100 you filled in. Add a row and it stores one more; remove rows and it stops storing them. (Python's lists behave in a similar way from the programmer's point of view, which is why Python has lists but no plain arrays.)

There are several dynamic data structures, and the linked list is the building block for most of them. In the next few lectures we will use it to build stacks and queues, and after that we will look at trees. For now, a linked list is just what it sounds like: a list that can grow and shrink as you add and remove things.

Here are four of the most common ones. We will only look at the first today.

- Linked lists store elements in a line, and insertion and deletion can happen at any point.
- Stacks are used by compilers and operating systems. Insertion and deletion only take place at one point, following the *last in, first out* (LIFO) principle.
- Queues are used for things like waiting lines and buffers. Data is inserted at one end (the tail) and removed at the other (the head), following the *first in, first out* (FIFO) principle.
- Trees are used to implement very efficient search and sort algorithms. Binary trees are the most basic type.

What is a node?

A linked list is built from little pieces called *nodes*, which are linked together. Each node contains two things:

1. the data that we want to store;
2. a pointer to the next node in the list.

If we want to store integers, the data is an integer. If we want to store bank customers, the data is a structure with an account number, a first name, a last name and so on. For now we will keep things as simple as possible and store a single character. A node is then defined like this:

```
struct listNode {  
    char data;  
    struct listNode *nextPtr;  
};  
typedef struct listNode ListNode;
```

This is a perfectly valid structure, even though it looks strange at first: the structure contains a pointer to another structure of its own type. It is called a *self-referential* structure. Note that it does not contain a complete `struct listNode` inside itself (that would be impossible, since it would never end); it contains only a pointer to one, and a pointer always has a known size. As usual, the `typedef` just saves us from writing `struct listNode` over and over; from now on we can write `ListNode`.

What happens in memory when we create a node?

```
ListNode a = {'h', NULL};
```

This creates a box called `a`, of type `ListNode`. That box has two boxes inside it: `a.data`, which holds the character `'h'`, and `a.nextPtr`, which holds `NULL` and so is not pointing at anything yet. The figure below shows this, with the cross in the second box meaning `NULL`.

```
struct ListNode {
    char data;
    struct ListNode *nextPtr;
};
typedef struct ListNode ListNode;

ListNode a = { 'h', NULL};
```

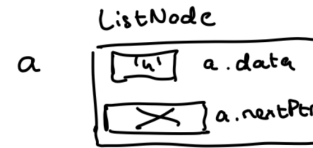


Figure 1: A node `a` of type `ListNode`. It has two members: `a.data` holds a character, and `a.nextPtr` holds `NULL` (drawn as a cross) and so points at nothing.

How many bytes does `a` take up? The character takes one byte, and the pointer is an address, which takes eight bytes on a typical modern computer. That makes at least nine. In practice the compiler usually adds padding between the members (as we saw with structures earlier in the course), so on most systems the whole node takes up 16 bytes.

The basic idea of a linked list is now easy to state. You create a node, then you create another one and link it to the first, then another one and link it to the previous one, and so on. If you want to remove a node, you break the links around it and throw it away. The list grows and shrinks as the program runs.

Linking two nodes together

Let us make this concrete with a small and rather contrived example. We define two nodes and link them:

```
struct ListNode {
    char data;
    struct ListNode *nextPtr;
};
typedef struct ListNode ListNode;

int main()
{
    ListNode firstNode = {'c', NULL};
    ListNode secondNode = {'k', NULL};

    firstNode.nextPtr = &secondNode;

    return 0;
}
```

The first line of main creates somewhere in memory a box called `firstNode`, with a 'c' in its first member and NULL in its second. The second line creates another box, somewhere else in memory, called `secondNode`, with a 'k' and NULL. At this point the two nodes are not connected in any way; they are just two separate boxes.

The third line is the one that links them. It takes the address of `secondNode` and stores it in the `nextPtr` member of `firstNode`. Suppose `secondNode` lives at address 904641. Then the empty box inside `firstNode` now has the number 904641 in it, and we draw this as an arrow from that box to `secondNode`. The figure below shows the situation after the three lines have run.

```

struct listNode {
    char data;
    struct listNode *nextPtr;
};
typedef struct listNode ListNode;

int main()
{
    ListNode firstNode = { 'c', NULL };
    ListNode secondNode = { 'k', NULL };
    firstNode.nextPtr = &secondNode;

    return 0;
}

```

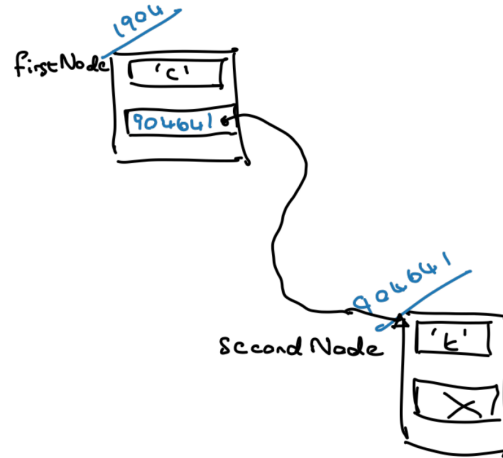


Figure 2: The state after `firstNode.nextPtr = &secondNode;`. The `nextPtr` member of `firstNode` holds the address of `secondNode`, drawn as an arrow. The `nextPtr` of `secondNode` is still NULL.

If we want a longer list, we create another node and make the `nextPtr` of the second node point to it. Then we create another and make the `nextPtr` of the third point to it, and so on. To reach any element of a list like this, we start at the first node and follow the pointers one at a time. This particular example, a list of single characters, is not very useful, but you get the idea.

You do not have to store single characters. Here is a second example in which each node stores a row and a column:

```
struct listNode {
    int row;
    int col;
    struct listNode *nextPtr;
};
typedef struct listNode ListNode;

int main()
{
    ListNode firstNode = {2, 1, NULL};
    ListNode secondNode = {2, 2, NULL};

    firstNode.nextPtr = &secondNode;

    return 0;
}
```

You can think of this as the record of a robot moving through a maze. The robot started at row 2, column 1, and then moved to row 2, column 2. Which direction did it move in? Along the row, to the right, since only the column changed. If the robot takes more steps, we simply add more nodes. A list like this is a natural way to record the path of a robot, because we cannot know in advance how many steps it will take.

The start pointer and the arrow notation

Every linked list needs a way for the program to find its first node. This is a pointer that we usually call `startPtr` (or just `start`), which holds the address of the first node. When the list is empty, there is no first node, so `startPtr` is `NULL`. When the first node is created, `startPtr` is made to point to it.

If we lose the start pointer, we lose the whole list, because it is the only way into it.

Here is how we would set this up for our row-and-column example:

```
ListNode *start;
ListNode firstNode = {2, 1, NULL};
start = &firstNode;
ListNode secondNode = {2, 2, NULL};

firstNode.nextPtr = &secondNode;
```

The first line creates a box called `start`. What does this box hold? An address, so it takes eight bytes, and the address is that of a `ListNode`. The line `start = &firstNode;` puts the address of `firstNode` in that box. If `firstNode` lives at address 16, then 16 goes into `start`, and we draw an arrow from `start` to `firstNode`.

```

struct listNode {
    int row;
    int col;
    struct listNode *nextPtr;
};
typedef struct listNode ListNode;

int main()
{
    ListNode *start;
    ListNode firstNode = {2, 1, NULL};
    start = &firstNode;
    ListNode secondNode = {2, 2, NULL};

    firstNode.nextPtr = &secondNode;
    (*start).col = 3;    // start -> col = 3 ;
    return 0;
}

```

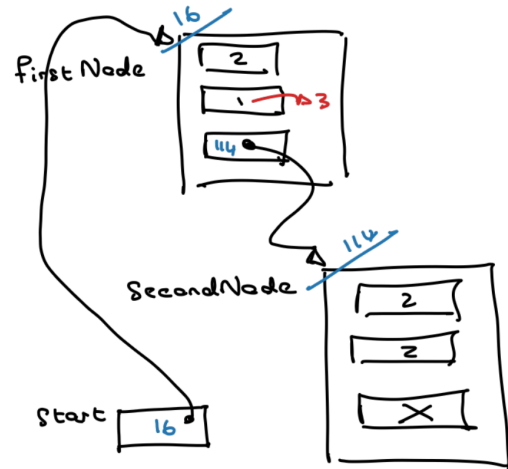


Figure 3: The same two nodes with a start pointer added. The pointer start holds the address 16 of firstNode, and the nextPtr of firstNode holds the address 114 of secondNode. The red 3 shows the effect of the assignment `start->col = 3;`.

Now suppose we want to change the column of the first node to 3, but we want to do it *through* the pointer start. We cannot write `start.col`, because start is a pointer and not a structure. We have to dereference it first, which means going to the box that it points at, and then pick out the member:

```

(*start).col = 3;

```

The brackets are needed because `.` binds more tightly than `*`. This is such a common thing to write that C has a shorthand for it, the arrow notation:

```

start->col = 3;

```

These two lines are exactly the same. The arrow means “go to the box that this pointer points at, and then open up this member”. You will see it everywhere in code that uses linked lists, because we are constantly working with pointers to nodes and changing the nodes that they point to. It is worth making sure that you are completely comfortable with it before continuing.

Creating nodes while the program runs

Everything we have done so far was a bit silly, because we created every node by hand in the code. We had to know, when writing the program, how many nodes there would be and what would go into them. That is the very thing we wanted to get away from.

In a real program, such as the one for the bank, the list starts out with nothing in it. When the first customer arrives, the program creates a node for them *while it is running*. When the second customer arrives, the program creates another node, and makes sure that the first node points to it. And so on. To create a node while the program runs, we use `malloc`, which reserves enough

bytes for one node; and to destroy a node that is no longer needed, we use `free`. We always use `sizeof` to say how many bytes a node needs:

```
ListNode *newPtr = malloc(sizeof(ListNode));
```

This is the real reason `malloc` exists. In the examples above we could have written `ListNode a = {7, NULL}`; and then pointed at `a`, but that only works if we know when writing the program that there will be a node with a 7 in it. If instead the user types in the value while the program runs, and we create a node inside a loop, a local variable would be the same box on every pass through the loop, and each new value would overwrite the previous one instead of creating a new node. With `malloc`, each call gives you a brand new box.

With a list in hand, there are three things that we typically want to do with it: search for a value, insert a new node, and delete a node. The rest of the lecture goes through these three operations in turn. First, the picture of a list that we will use below. It stores integers, and has a start pointer, four nodes, and `NULL` in the last node.

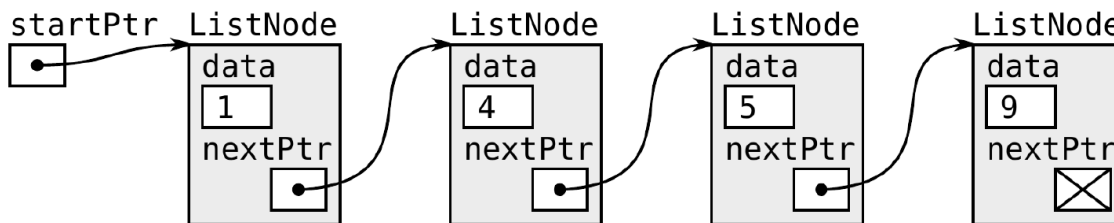


Figure 4: A linked list storing the integers 1, 4, 5 and 9. The pointer `startPtr` points at the first node, each node points at the next one, and the last node holds `NULL` (drawn as a cross).

Searching a list

Suppose that we want to know whether the value 5 is in the list above, and if so, where. The code is short:

```
value = 5;
currentPtr = startPtr;
while (currentPtr != NULL && value != currentPtr->data) {
    currentPtr = currentPtr->nextPtr;
}
```

The best way to understand this is to step through it with the picture below. Each node in the picture has an address, which we write next to it. Suppose the nodes live at addresses 9046, 114, 9011 and 604, in that order.

At the start, `startPtr` holds 9046, the address of the first node. The line `currentPtr = startPtr;` creates another pointer and copies that address into it, so both pointers hold 9046 and both point at the first node.

Now we reach the loop condition, which has two parts. The first checks that `currentPtr` is not `NULL`; it isn't. The second checks whether the value we are looking for differs from the data in

```

value = 5;
currentPtr = startPtr;
while ( currentPtr != NULL && value != currentPtr->data ) {
    currentPtr = currentPtr->nextPtr;
}

```

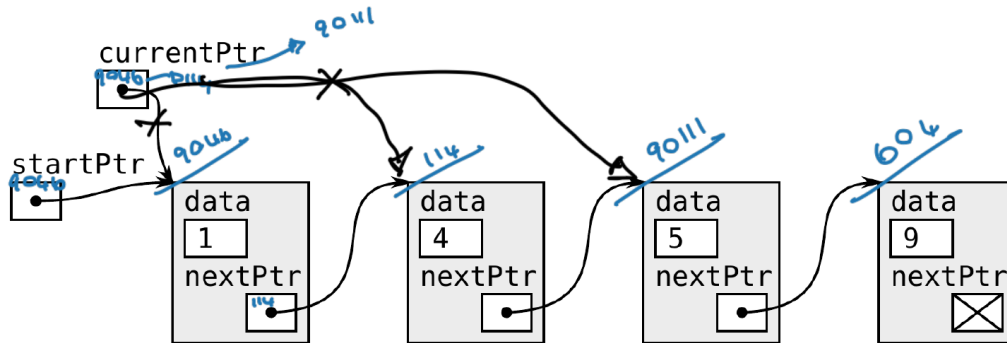


Figure 5: Searching for the value 5. The pointer `currentPtr` starts at the first node (address 9046), moves to the second (114) and finally to the third (9011), where it stops because the data is 5.

the node that `currentPtr` points at. Here `currentPtr->data` means: go to the box at address 9046 and open up its data member. It holds 1, and 1 is not 5, so the second part is true as well. Both parts being true, we enter the loop.

The one line in the loop body is where the magic happens:

```
currentPtr = currentPtr->nextPtr;
```

Read it from right to left. `currentPtr->nextPtr` says: go to the box at address 9046 and read its `nextPtr` member. That member holds 114. We take that 114 and store it in `currentPtr`. Now `currentPtr` holds 114 and, if we draw the arrow, it points at the second node. The pointer has moved one step along the list.

We go back to the condition. The data in the node at 114 is 4, which is not 5, so we go round again, and `currentPtr` becomes 9011. This time the data is 5. The second part of the condition is false, so the loop stops.

When the loop ends, which value does `currentPtr` hold? Not 5: it holds 9011, the *address* of the node that contains the 5. It is easy to mix these up.

What if the value is not in the list? Then `currentPtr` keeps moving along until it gets to the last node, whose `nextPtr` is NULL. That NULL is copied into `currentPtr`, and the first part of the condition now fails, so the loop stops. This is also why the condition checks for NULL first. The `&&` operator evaluates its left side first, and if that is false it does not evaluate the right side. Without this, the program would try to evaluate `currentPtr->data` through a null pointer, which would crash it.

So the loop always ends, whether or not the value was found. After it, we have to check which

case occurred before doing anything with `currentPtr`: if it is `NULL` the value is not in the list; otherwise `currentPtr->data` is the value we were looking for.

Notice what this tells us about linked lists. To get to the third node we had to walk through the first two. There is no way to jump straight to element number i as you can with an array. For a list with n nodes, a search may have to look at all n of them.

Inserting a node

Now we turn to the question of where a list comes from. The answer is that we build it one node at a time. We start with a `startPtr` that is `NULL`; then we create a node with `malloc`, make `startPtr` point at it, and so on.

Inserting a node into the middle of a list is the trickiest case, and it shows how nodes are created. Suppose we have the list 1, 4, 5, 9 and we want to insert a new node holding 7 between the 5 and the 9. We assume that an earlier search has left us with two pointers: `previousPtr`, pointing at the node with the 5, and `currentPtr`, pointing at the node with the 9. The new node must go between them.

```
newPtr = malloc(sizeof(ListNode));
newPtr->data = 7;
newPtr->nextPtr = NULL;
previousPtr->nextPtr = newPtr;
newPtr->nextPtr = currentPtr;
```

(Strictly, a 7 between a 5 and a 9 keeps the list in increasing order, which is why this position is a sensible one to pick.) The figure below shows the state of things after all of these lines have run. Let us go through them one by one.

The first line calls `malloc`, which reserves enough bytes for one `ListNode` somewhere in memory. Say these bytes start at address 904. The bytes have not been set to anything, so for now there is rubbish in them. `malloc` returns the address 904, which we store in `newPtr`. In the figure, `newPtr` has an arrow to this new box.

The second line goes to the box that `newPtr` points at, opens up its `data` member and puts a 7 in it. The third line goes to the same box and puts `NULL` in its `nextPtr` member. (Strictly speaking, we are about to overwrite this, so the program would work without the line. But it is good practice to give every new node a `NULL` link; if we were adding the node at the very end of the list, that `NULL` is exactly what we want.)

Now come the two lines that change the list. The fourth line, `previousPtr->nextPtr = newPtr;`, goes to the node with the 5 and replaces the address in its `nextPtr` member (the address of the 9) with 904, the address of the new node. The old arrow from the 5 to the 9 no longer exists; the 5 now points at the new node. We have not lost the 9, because `currentPtr` still points at it.

The last line, `newPtr->nextPtr = currentPtr;`, goes to the new node and puts the value of `currentPtr` in its `nextPtr` member. `currentPtr` holds the address of the node with the 9, so

```

✓ newPtr = malloc( sizeof( ListNode ) );
✓ newPtr->data = value7;
✓ newPtr->nextPtr = NULL;
✓ previousPtr->nextPtr = newPtr;
  newPtr->nextPtr = currentPtr;

```

ListNode * currentPtr;

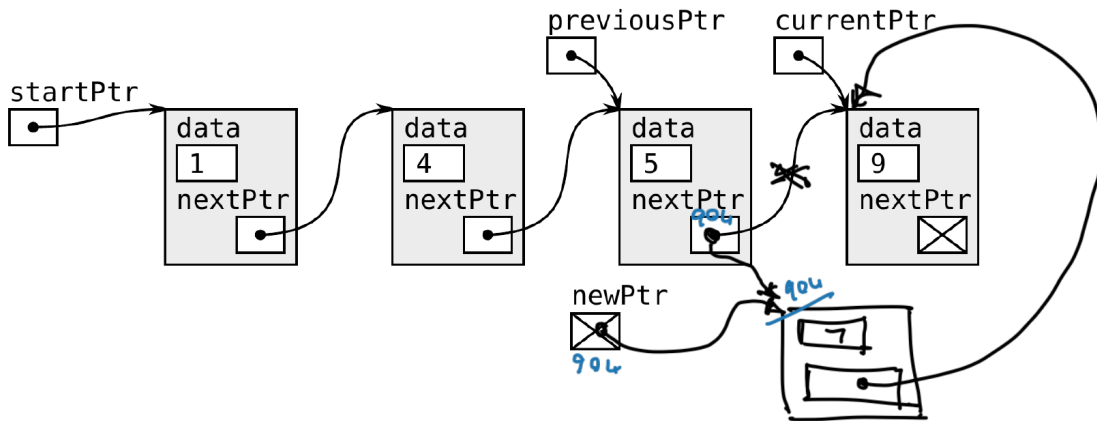


Figure 6: Inserting a node holding 7 between the nodes holding 5 and 9. The new node was created at address 904 and newPtr points at it. The crossed-out arrow is the old link from the 5 to the 9; the new links go from the 5 to the new node, and from the new node to the 9.

we copy that address into the new node, and the new node now points at the 9. Following the arrows from startPtr, we now go through 1, 4, 5, 7 and 9, and then reach NULL.

If this confuses you, the most important advice is to follow the boxes. A pointer is a box with an address in it. An arrow is how we draw the fact that there is an address inside the box. When you see $\rightarrow$, go to the box that lives at that address. Each line of code then just takes a value from one box and copies it into another.

The last two lines can be done in either order here, because currentPtr still holds the address of the 9. But be careful: without a pointer like currentPtr, the order would matter. If you redirected previousPtr->nextPtr first and had no other pointer to the 9, the rest of the list would be unreachable and lost. The safe habit is to make the new node point at its successor first, and only then redirect the previous node.

Inserting at the start

Inserting at the start of the list is a special case, because there is no previous node. Instead, we make the new node point at the old first node, and then make startPtr point at the new node:

```

newPtr->nextPtr = startPtr;
startPtr = newPtr;

```

This also works when the list is empty. Then startPtr is NULL, so the new node's nextPtr becomes NULL, which is correct, and startPtr now points at the one and only node.

Deleting a node

Deleting is the opposite of inserting. Suppose a customer has left the bank and we want to remove their node, the one holding 5, from the list 1, 4, 5, 9. We first search for it, and this time we end up with `previousPtr` pointing at the node before the one we want to delete (the 4), and `currentPtr` pointing at the node we want to delete (the 5). Then:

```
tempPtr = currentPtr;  
previousPtr->nextPtr = currentPtr->nextPtr;  
free(tempPtr);
```

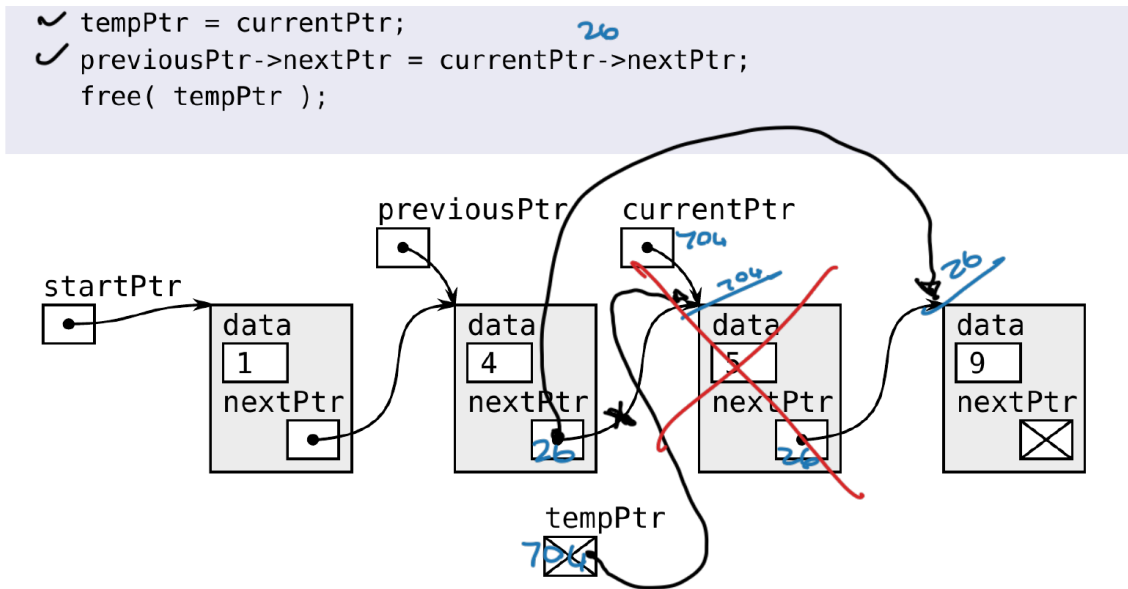


Figure 7: Deleting the node holding 5. Suppose it lives at address 704 and the node holding 9 lives at address 26. The `nextPtr` of the node holding 4 is changed from 704 to 26, so the list skips the 5. The memory of the removed node is then released.

Again, we take the lines one at a time. In the first, `tempPtr = currentPtr;`, we copy the address of the node to delete (704) into a third pointer, `tempPtr`. Both now point at that node.

The second line looks complicated, but we read it from right to left. `currentPtr->nextPtr` goes to the node at 704 and reads its `nextPtr` member, which is the address of the node after it. Say that is 26, the address of the node holding 9. The left side, `previousPtr->nextPtr`, is the `nextPtr` member of the node with the 4. We put 26 in it. The node with the 4 now points directly at the node with the 9, and the node with the 5 is no longer part of the list: nothing points at it.

The last line, `free(tempPtr);`, tells the operating system that the memory at address 704 is no longer needed. The node has been removed from the list *and* its memory has been released; the customer has left the bank.

The order matters. We must change the links before we free the node, and after `free` we must never touch that node again, whichever pointer we use. The pointer `tempPtr` is, strictly speaking,

not necessary here: we could have written `free(currentPtr);` as the last line and skipped the temporary pointer. Using `tempPtr` is a habit that makes it clearer what we are freeing.

Deleting the first node

As with insertion, deleting the first node is a special case, because there is no previous node. We make `startPtr` point at the second node, and free the old first node:

```
tempPtr = startPtr;
startPtr = startPtr->nextPtr;
free(tempPtr);
```

If the list has only one node, its `nextPtr` is `NULL`, so after this `startPtr` is `NULL` and the list is empty again, as it should be.

Linked lists compared with arrays

Anything you can store in a linked list you can also store in an array, so why bother? The main reason is that a linked list is dynamic. Its size can change with the requirements of the program, while the size of an array is fixed.¹ A linked list is also useful when you do not know in advance how many elements you will have. An array can get full; a linked list can almost always grow to hold more data, because it only needs one more node's worth of memory at a time. If you make an array big enough to be safe, you may waste a lot of memory, or even run out of it; a linked list takes only as much as it needs.

The comparison is not all in favour of linked lists. Searching a linked list means following pointers from the start, so you cannot jump to element number 500 directly as you can with an array. Each node also stores a pointer in addition to its data, so a linked list uses more memory per element. And inserting into the middle of an array means moving all the elements after the insertion point, whereas inserting into a linked list means changing two pointers (once you have found the right position, which takes a search).

When would you insert in the middle at all? Often you would not: in the maze example, you would always add a new node at the end. Stacks and queues, which we will see next, always add and remove at one end or the other, which makes them very efficient. But if you keep a list of students sorted by student number, then a new student has to go in at the right place for the list to stay sorted, and that means inserting in the middle.

It is also fair to say that it is rare to write a linked list of your own in a real program. Real programs usually use a data structure written by someone else and tailored to the problem. But linked lists are worth knowing for two reasons. They bring structures, pointers and `malloc` together in one place, so that if you understand them, you understand all three. And the same building blocks, nodes with pointers to other nodes, are what trees are made from.

¹Strictly, a block reserved with `malloc` can be resized with the standard function `realloc`, which finds a bigger block, copies the old contents across and releases the old block. This is how growable arrays work, such as Python's `list` or C++'s `vector`. They keep instant access by index, and they usually double their size when full so that the copying happens rarely. A linked list is a different trade-off, not the only way to get a list that grows.

Summary

A linked list is a linear collection of nodes connected by pointers. Each node stores some data and a pointer to the next node, and a pointer such as `startPtr` identifies the first node. The `nextPtr` of the last node is `NULL`, and an empty list is one where `startPtr` is `NULL`.

The essential node definition is:

```
struct ListNode {
    char data;
    struct ListNode *nextPtr;
};
typedef struct ListNode ListNode;
```

To search, we set a pointer to `startPtr` and keep moving it to the `nextPtr` of the node it points at, until we find the value or the pointer becomes `NULL`. To insert, we create a node with `malloc`, fill it in, and change two pointers: the previous node must point at the new node, and the new node must point at the next one. To delete, we make the previous node point at the node after the one being deleted, and then release the removed node with `free`. Insertion and deletion at the start of the list are special cases, because they change `startPtr`.

Common mistakes to look out for are:

- Dereferencing a null pointer, for example by using `currentPtr->data` when `currentPtr` is `NULL`
- Losing the rest of the list by changing a pointer before saving where it pointed
- Freeing a node too early, or using a node after it has been freed
- Forgetting to update `startPtr` when changing the first node
- Forgetting the empty-list case

The examples in this lecture are code snippets rather than complete programs. To see all of these operations working together, try to write your own full version from scratch that creates a linked list from nothing and then search, insert and delete nodes from the list.

Exercises

1. A `ListNode` stores a character and a pointer. Why can a structure contain a pointer to its own type, but not a member of its own type?
2. In the search loop, what would go wrong if the two parts of the condition were swapped, so that it read `value != currentPtr->data && currentPtr != NULL`?
3. After the search loop ends with the value found, what does `currentPtr` hold: the value, or something else? What if the value was not found?
4. Draw the list 1, 4, 5, 9 with addresses of your choosing. Step through the insertion of 7 between 5 and 9, redrawing the boxes after each line.
5. Write the code to insert a new node at the *end* of a list, assuming the list is not empty. What does your code need to find first?
6. Write the code to delete the node with a given value, including the case where it is the first node.
7. Why is it important to call `free` on a node that has been removed from a list? What happens if the program never does this, and runs for a long time?